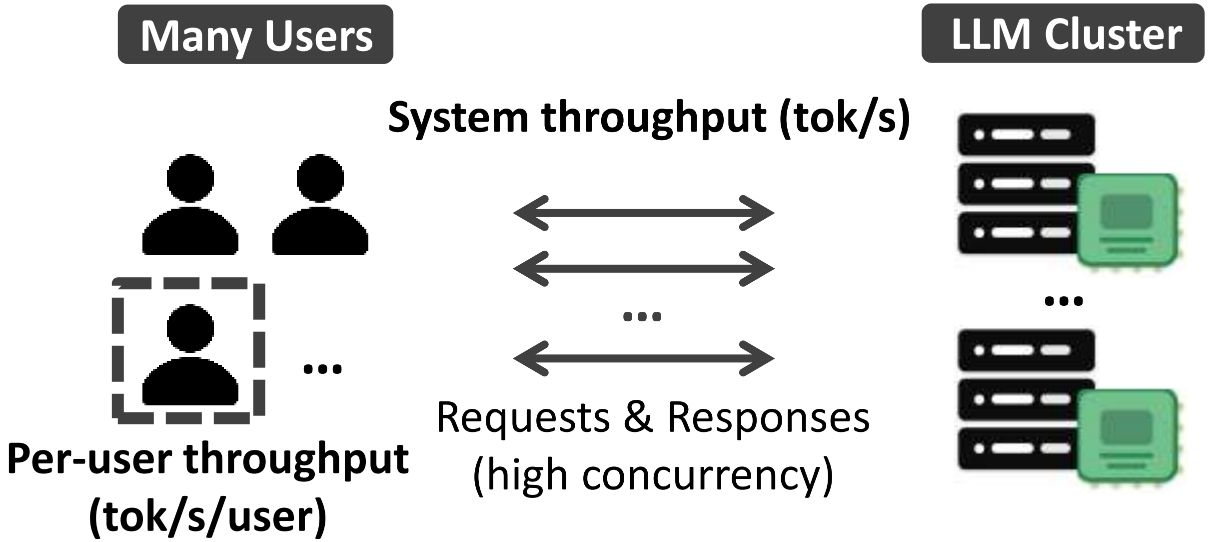
SHyLA: 3D-Stacked NVM-DRAM Hybrid LLM-Inference Architecture Exploiting Data and Memory Heterogeneity

Liu He[1], Fuyao Zhou[1], Cheng Peng[1], Shunan Dong[1], Ziming Zhang[1], Huazhong Yang[1], Yongpan Liu[1], Guowei Zhang[2], Hongyang Jia[1]

[1]Tsinghua University, Beijing, China
[2]Huawei Technologies Co., Ltd., Shanghai, China

Cloud LLM Inference: Memory-Bound

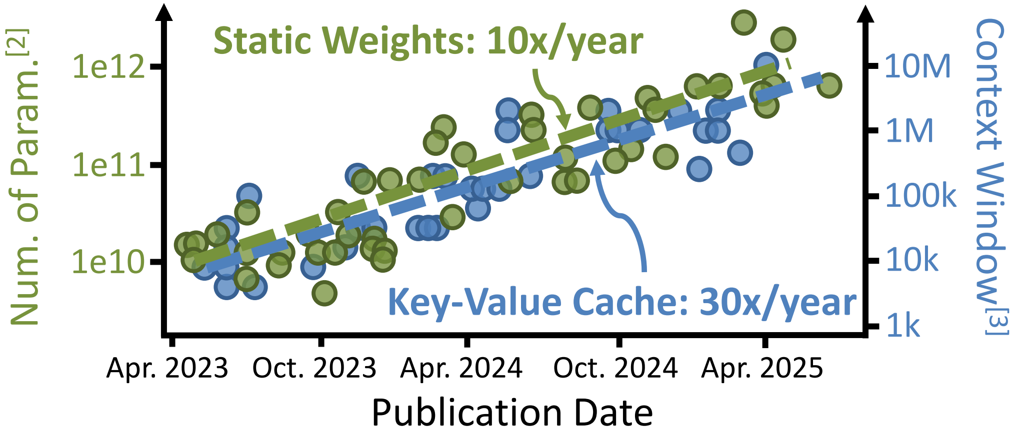
- **Scenario:** Multi-user cloud LLM inference



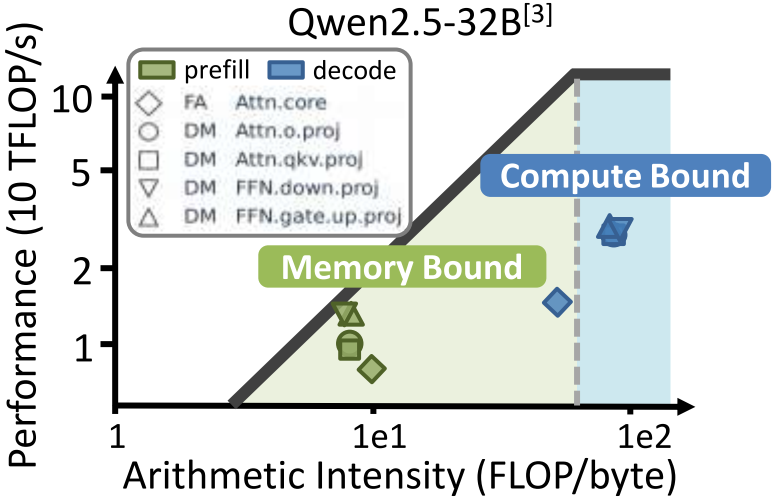
service-level objective (SLO): 25 tok/s/user[1]

- **Optimization Goal:** Maximize **system throughput** while satisfying **per-user SLO** constraints

- Growing static weights and key-value cache demand larger **memory capacity**



- Decode phase demands high **bandwidth**



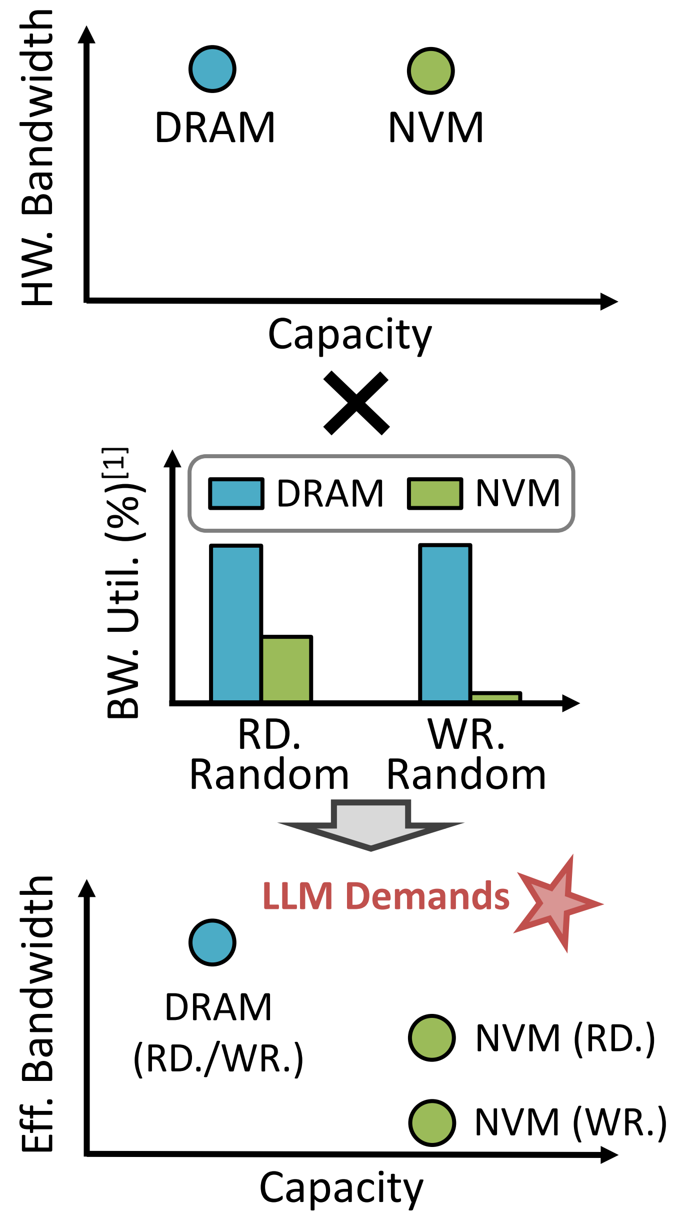
[1] MLPerf Inference v5.0 Advances Language Model Capabilities for GenAI – MLCommons
[2] Data on AI Models | Epoch AI
[3] LLMs now accept longer inputs, and the best models can use them more effectively | Epoch AI
[4] Wang et al., A Systematic Characterization of LLM Inference on GPUs, arXiv 2025

Prior Work Try to Enhance Memory, But…

- **Ignore Memory Heterogeneity**
  - DRAM-only system
    - **√ High effective bandwidth**
    - **✗ Capacity-limited for large models and key-value cache**
  - Non-Volatile-Memory (NVM)-only system
    - **√ Large capacity**
    - **✗ Poor write performance and endurance**
- **Ignore Data Heterogeneity**
  - Targeting general workloads with **random access** patterns, existing NVM-DRAM hybrid systems suffer from **poor NVM bandwidth utilization**
  - Overlook LLM-specific opportunity such as **streamable reads** and **data heterogeneity**

Missing Opportunity

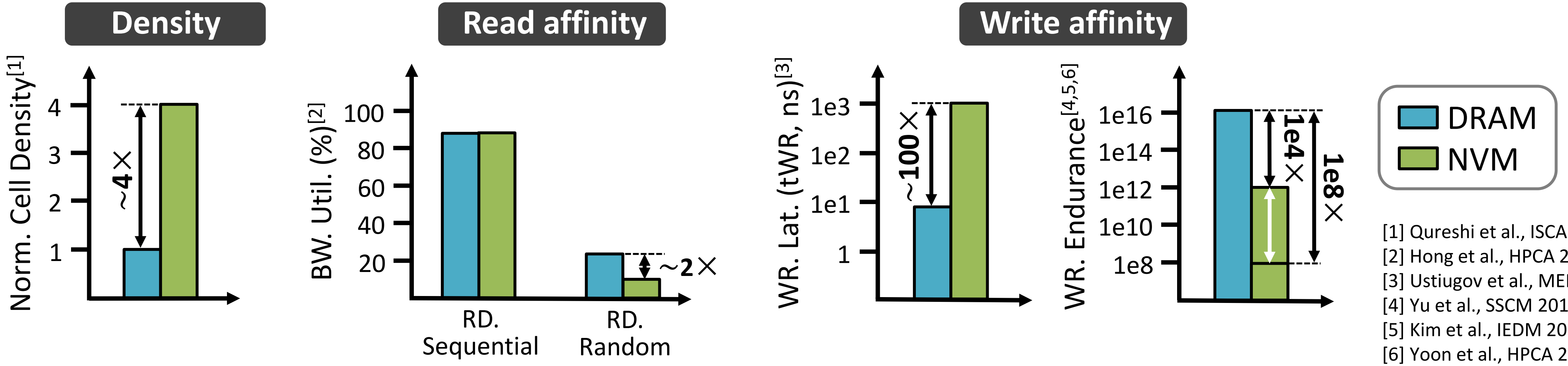
Jointly exploit **memory device** and **LLM data heterogeneity**



[1] Hong et al., Bandwidth-Effective DRAM Cache for GPUs with Storage-Class Memory, HPCA 2024

Memory Device Heterogeneity

- **Memory Device:** DRAM & NVM (e.g. Phase Change Memory)
- **Memory device heterogeneity**
  - Memory density (Capacity)
  - Read affinity: random & sequential access
  - Write affinity: write latency & write endurance



[1] Qureshi et al., ISCA 2009
[2] Hong et al., HPCA 2024
[3] Ustiugov et al., MEMSYS 2018
[4] Yu et al., SSCM 2016
[5] Kim et al., IEDM 2016
[6] Yoon et al., HPCA 2011

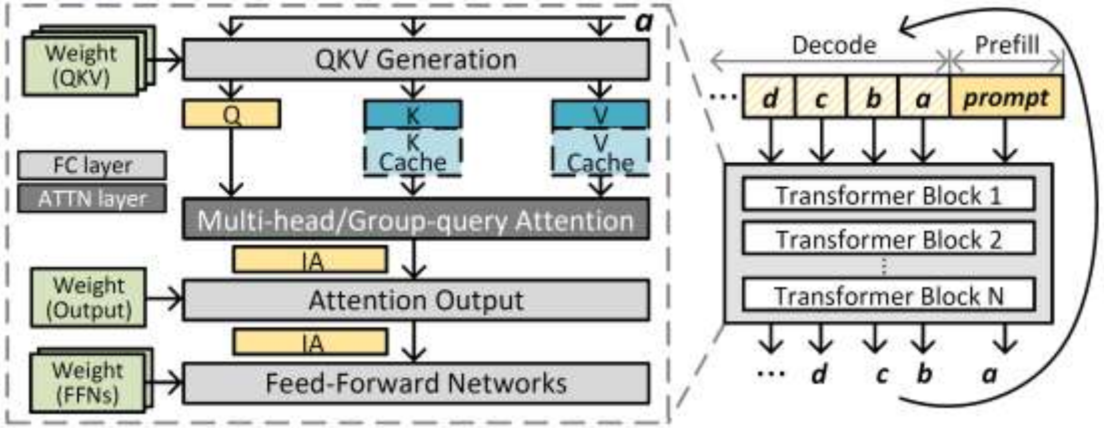
Memory Heterogeneity

DRAM: low density, balanced read/write performance, high write endurance
NVM: high density, sequential-read friendly, poor write performance & endurance

LLM Data Heterogeneity

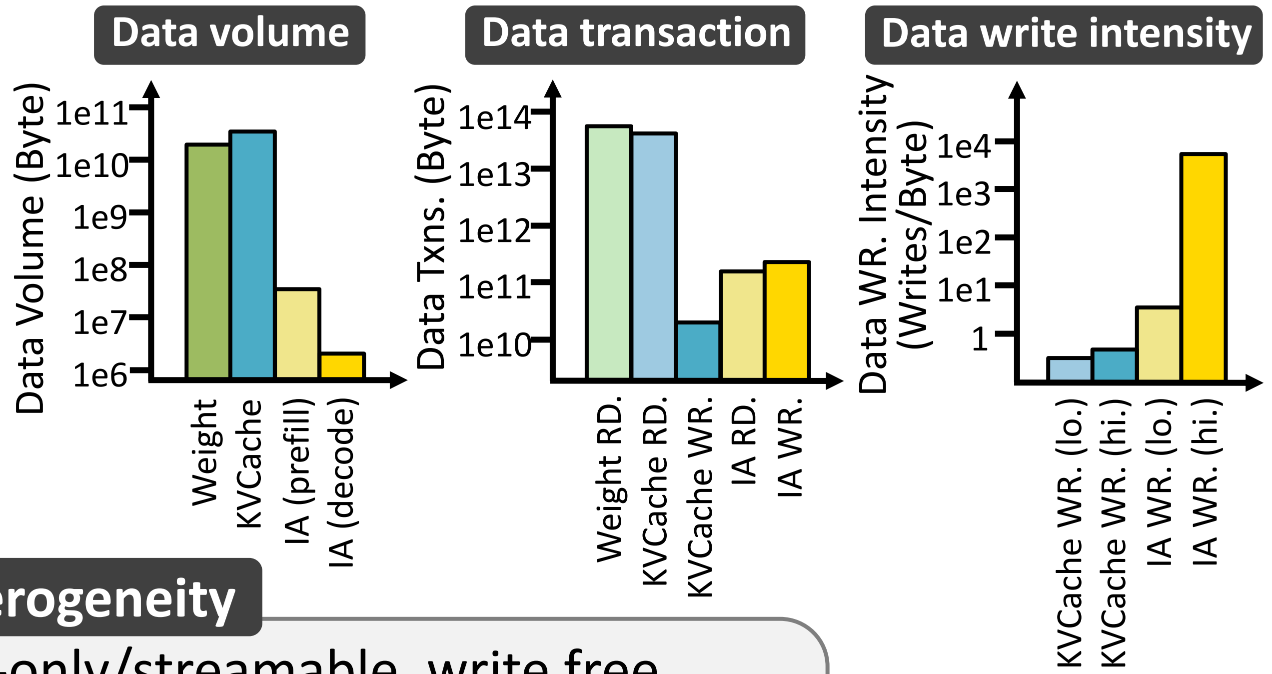
LLM data category

- static weights (Weight)
- key-value cache (KVCache)
- input activations (IA)



LLM data heterogeneity

- Data volume
- Data transaction
- Data write intensity



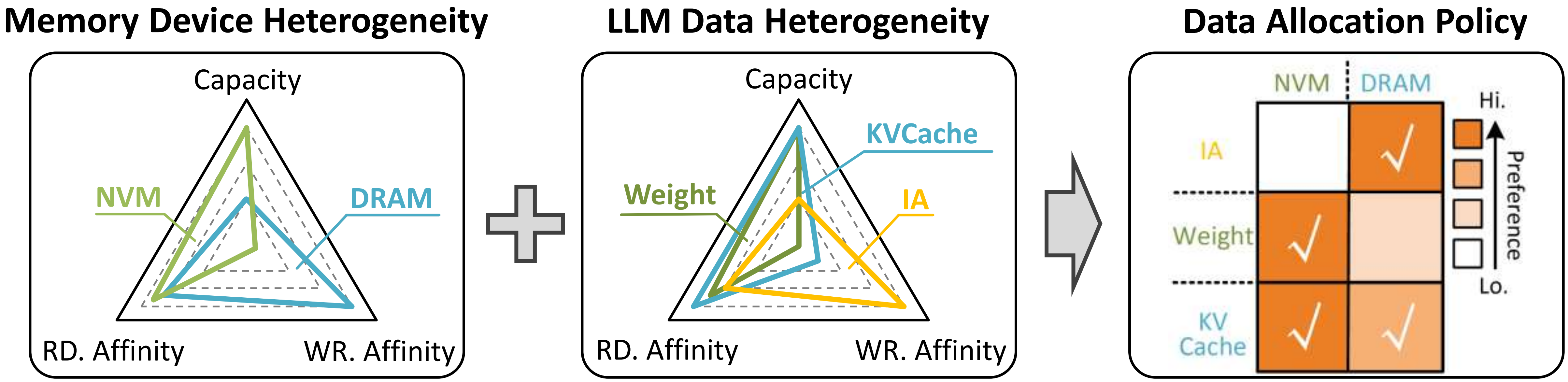
Data Heterogeneity

Weight: large volume, read-only/streamable, write free
KVCache: large volume, read-heavy/streamable, write infrequent
IA: small volume, read/write-intensive

*Grok1 314B (MHA+MoE) (1024,2048)
** Results obtained from simulation

The Insight: Dual-Heterogeneity Match

- Aligning LLM data heterogeneity with guides data allocation policy



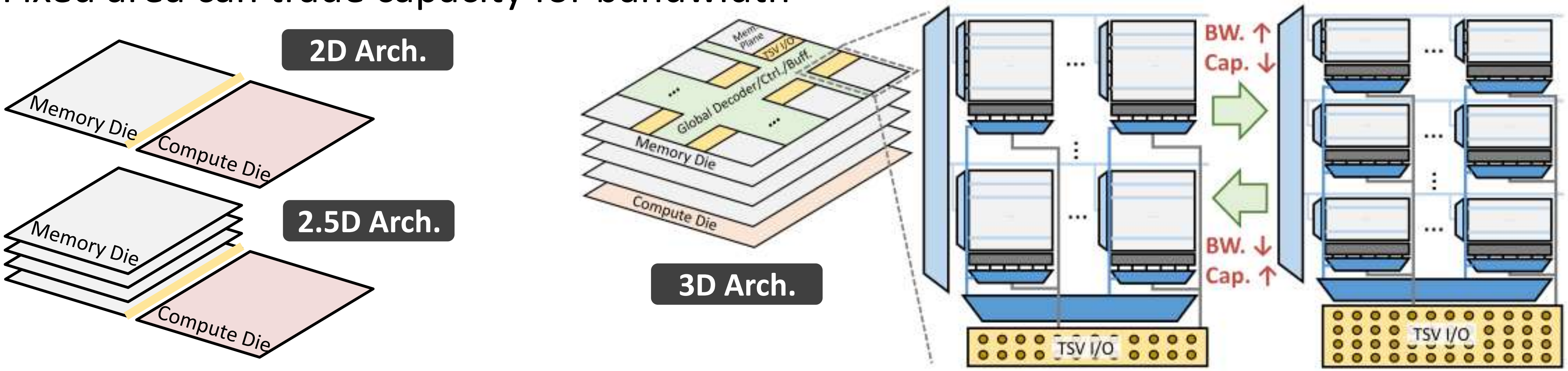
Data Allocation Policy

IA: DRAM-only
Weight: NVM-only
KVCache: NVM-first, DRAM spillover

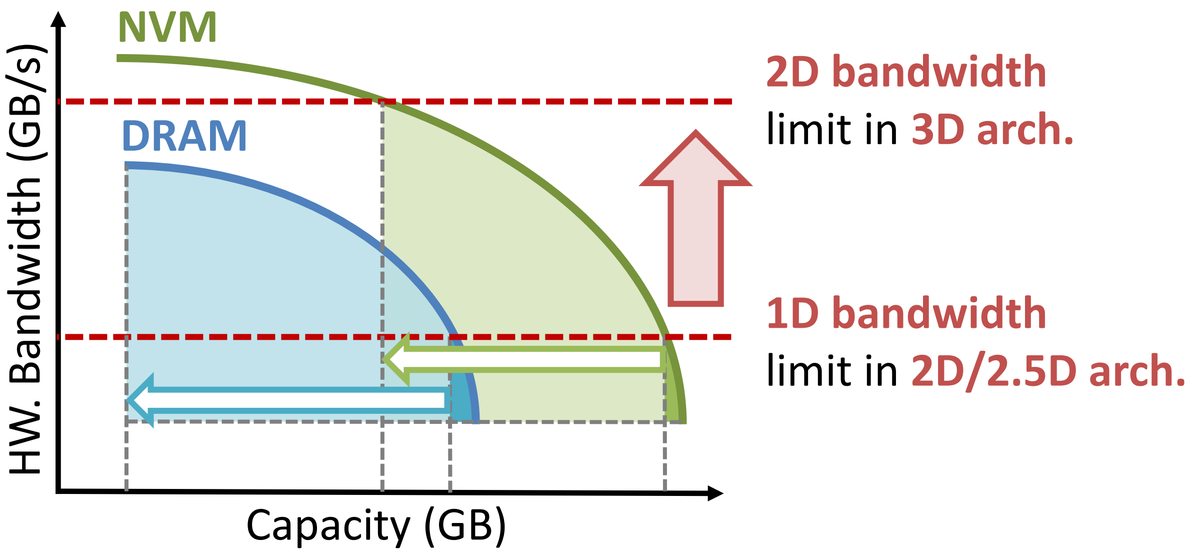
Dual-heterogeneity match — but can NVM provide sufficient read bandwidth?

3D Stacking Expands the Bandwidth-Capacity Trade-off

- Fixed area can trade capacity for bandwidth

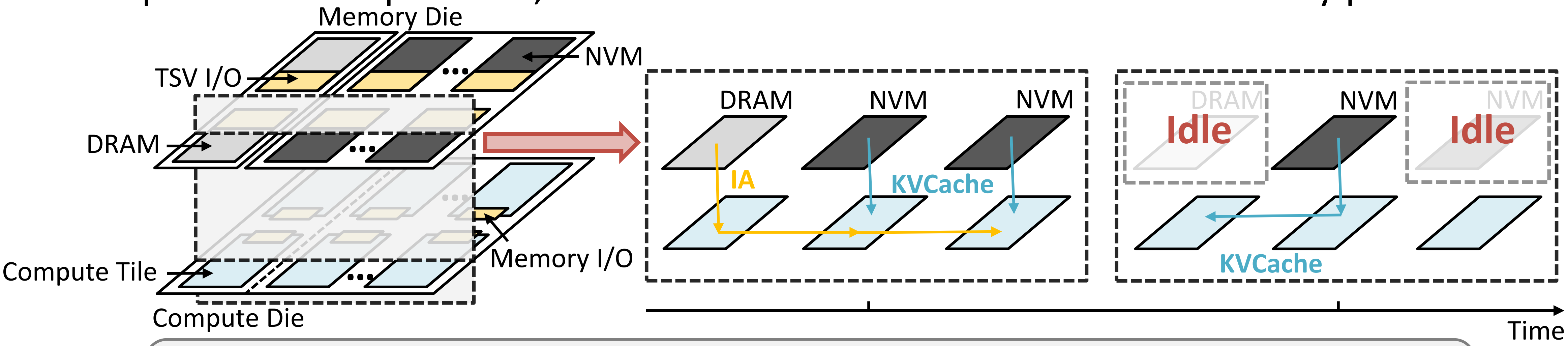


- With 3D stacking, bandwidth can scale in 2D, further expanding the design space beyond 2D/2.5D architectures
- Dense NVM provides higher bandwidth with only modest capacity loss, which fits massive and streamable reads in LLM inference



Challenge 1: Making Areal Bandwidth Effective

- 3D stacking provides high areal hardware bandwidth, but effective bandwidth depends on utilization: **Effective Bandwidth = HW. Bandwidth × Bandwidth Utilization**
- LLM data is distributed across heterogeneous memory planes: IA in DRAM, while Weight / KVCache (mostly) in NVM
- Each compute tile needs both DRAM-resident IA and NVM-resident Weight / KVCache to complete the computation; uncoordinated accesses leave some memory planes idle



A dedicated dataflow is needed to **keep DRAM/NVM planes active, converting 3D-stacked areal hardware bandwidth into effective bandwidth**

Challenge 2: Identifying the Optimal Design Point

- **The combinatorial design space** of hybrid memory and deployment is enormous

Hybrid Memory Design Space (HW.)

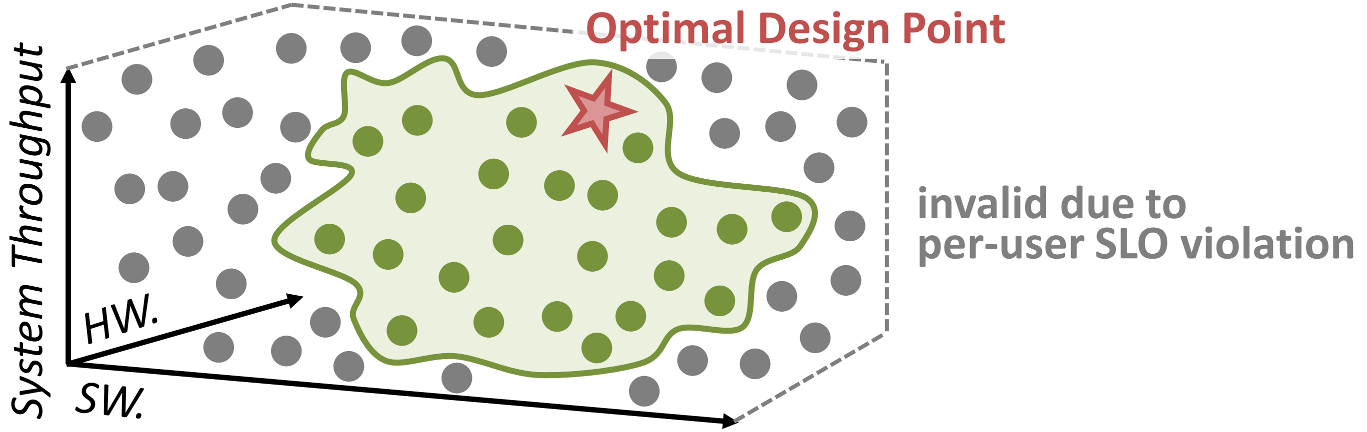
NVM-DRAM area ratio
×
Per-memory bandwidth-capacity trade-off

×

Deployment Design Space (SW.)

Batch size
×
PD aggregation/disaggregation × Parallelisms

- **The objective is constrained:** maximize system throughput under a per-user SLO



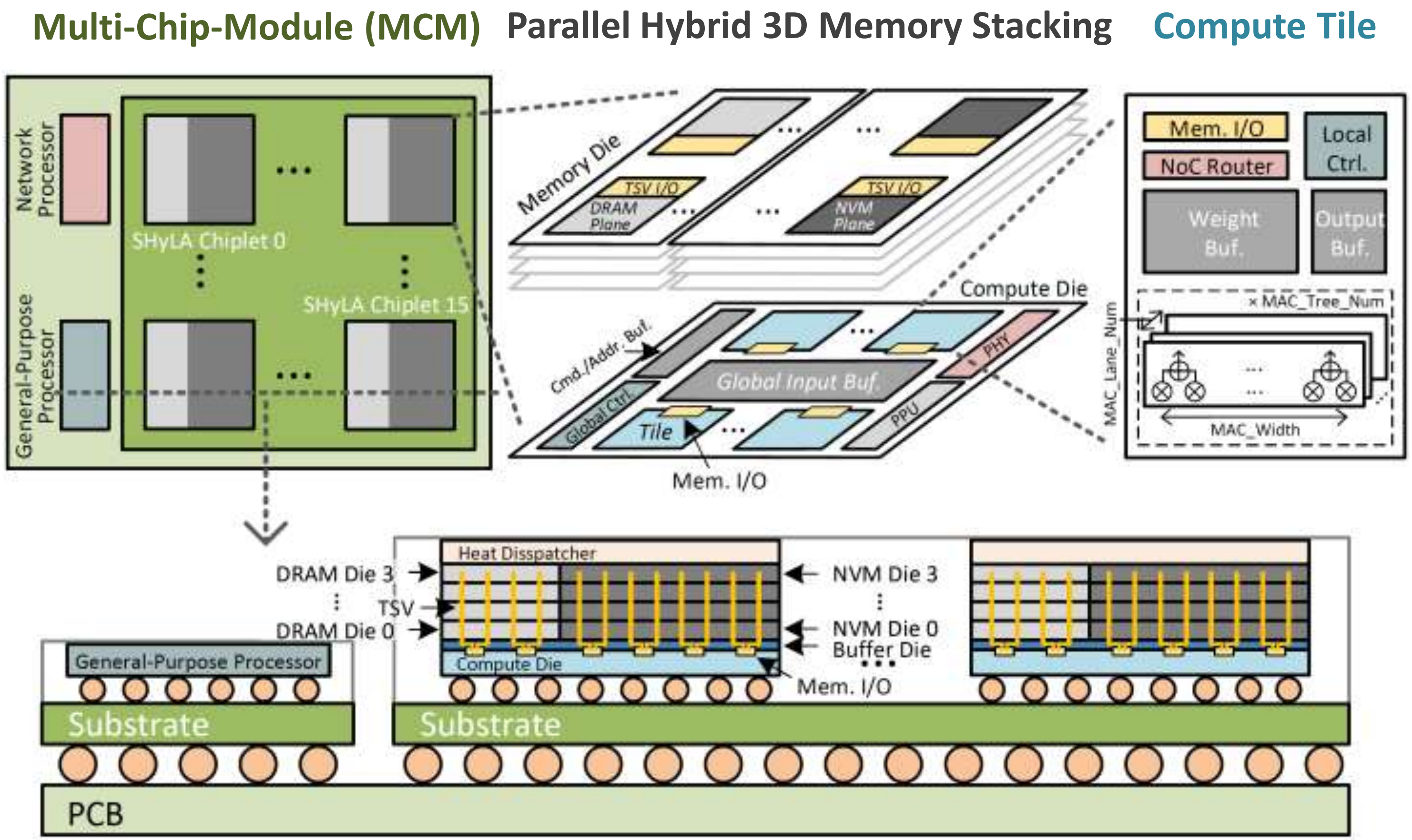
Need an **efficient design space exploration (DSE) methodology** to search the large constrained space and **surface design insights**

Outline

- Motivation
- **SHyLA Design**
  - **3D-Stacked NVM-DRAM Hybrid Memory Architecture Overview**
  - **Bandwidth-Utilization-Centric Dataflow**
  - **Two-Stage DSE**
- Evaluation
- Takeaways

SHyLA Architecture Overview

- Overview of 3D-stacked NVM-DRAM architecture



Multi-Chip-Module (MCM)

Multiple SHyLA chiplets integrated for scalable LLM inference

Parallel Hybrid 3D Memory Stacking

Side-by-side 4-Hi DRAM/NVM stacks above compute die (**die-to-wafer**)→ **independent bandwidth–capacity optimization**

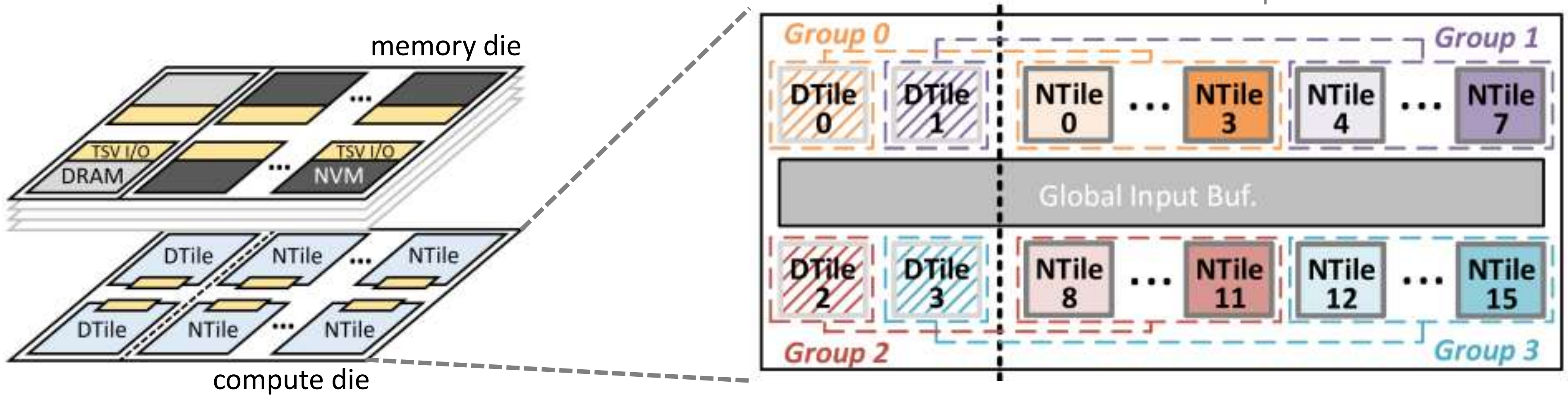
Compute Tile

20 homogeneous tiles per compute die, each physically mapped to a stacked memory plane

Compute Die: Hierarchical Tile Connectivity

- Memory-paired tile groups
  - Identical tiles, different memory associations: **DTile ↔ DRAM plane**; **NTile ↔ NVM plane**
  - Tile Groups pair DTiles/NTiles by relative position

*Take 4x DTiles & 16x NTiles as an example



Hierarchical Tile Connectivity

Two-level NoC **localizes dominant NVM ↔ DRAM traffic** while preserving full-chip connectivity

Level-1: Intra-group dedicated links

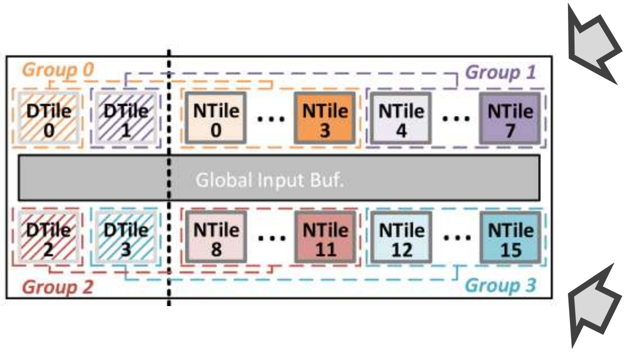
NTile(s) provide NVM-stored Weight/KVCache to DTile(s);
DTile(s) provide DRAM-resident data to NTile(s)

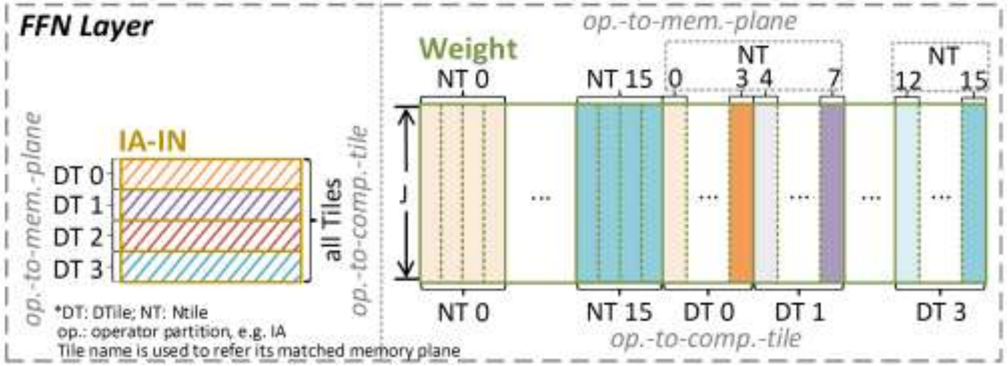
Level-2: Inter-group AXI fabric

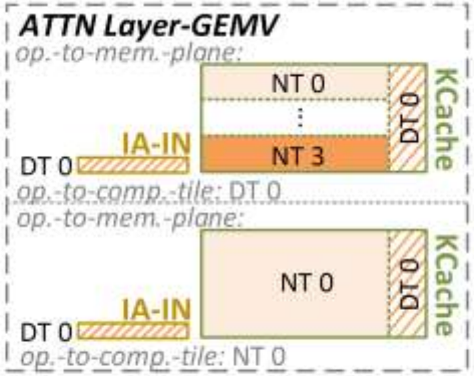
AXI handles occasional cross-group data redistribution

Bandwidth-Utilization-Centric Dataflow

- **Target:** Convert 3D-stacked areal hardware bandwidth into effective bandwidth through **memory-plane-aware layout** and **layer-specific execution**







FFN Layer

Weight cols partitioned across NVM planes;
In-group NVM planes also serve DTile Weight;
IA rows partitioned across DRAM planes

All tiles jointly compute one GEMM

ATTN Layer

NTile-mapped ATTN head: KVCache stored in the NVM plane above;
DTile-mapped ATTN head: KVCache partitioned across in-group NVM planes

Prefill phase: all tiles collaboratively compute one ATTN head;
Decode phase: each tile independently computes one ATTN head

Bandwidth-Utilization-Centric Dataflow

Memory-plane-aware layout and layer-specific execution **keep heterogeneous memory planes active** → higher areal bandwidth utilization

Two-stage DSE: Searching a Vast Design Space Effectively

☐ **Goal:** find the system-throughput-optimal design under per-user SLO constraints

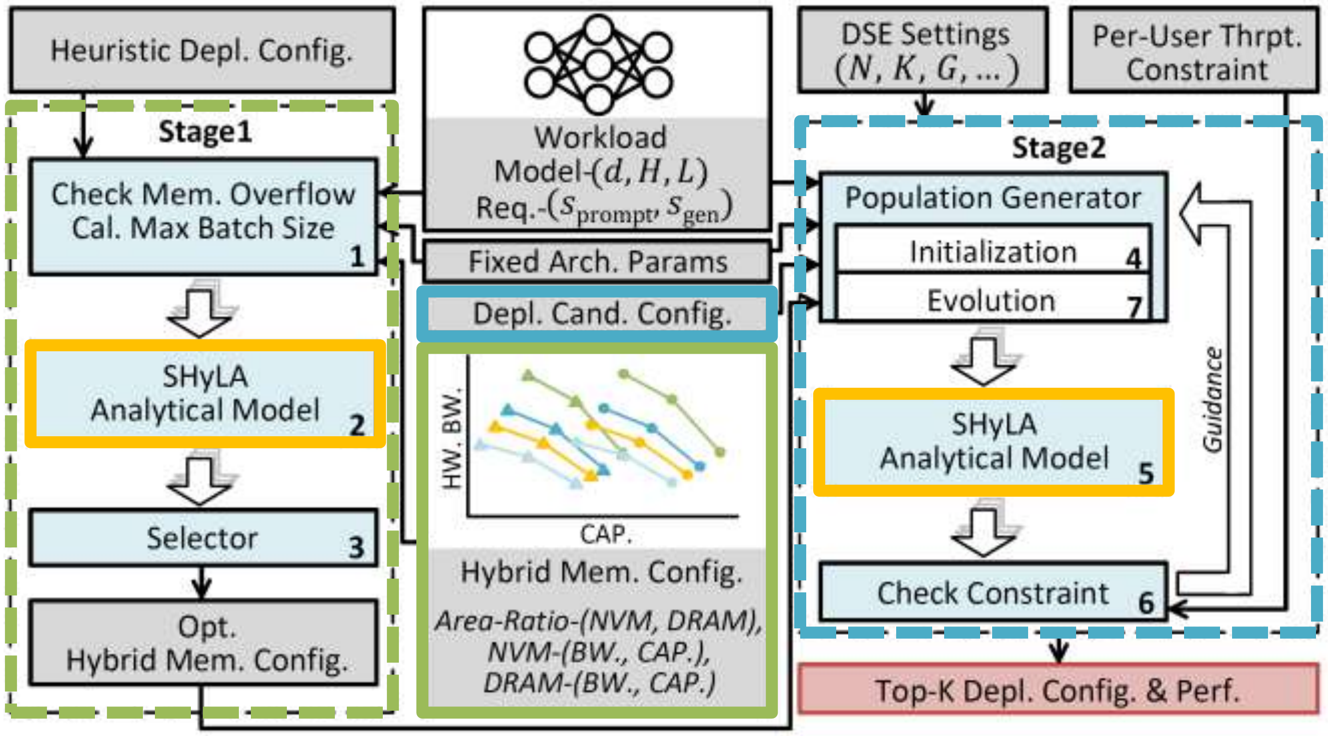
Hybrid Memory Design Space (HW.)

NVM-DRAM area ratio
×
Per-memory bandwidth-capacity trade-off

×

Deployment Design Space (SW.)

Batch size
×
PD aggregation/disaggregation × Parallelisms



Stage 1

System-throughput-oriented heuristic-based hybrid-memory-configuration determination

+

Stage 2

Per-user-throughput-constrained DSE with evolutionary algorithm

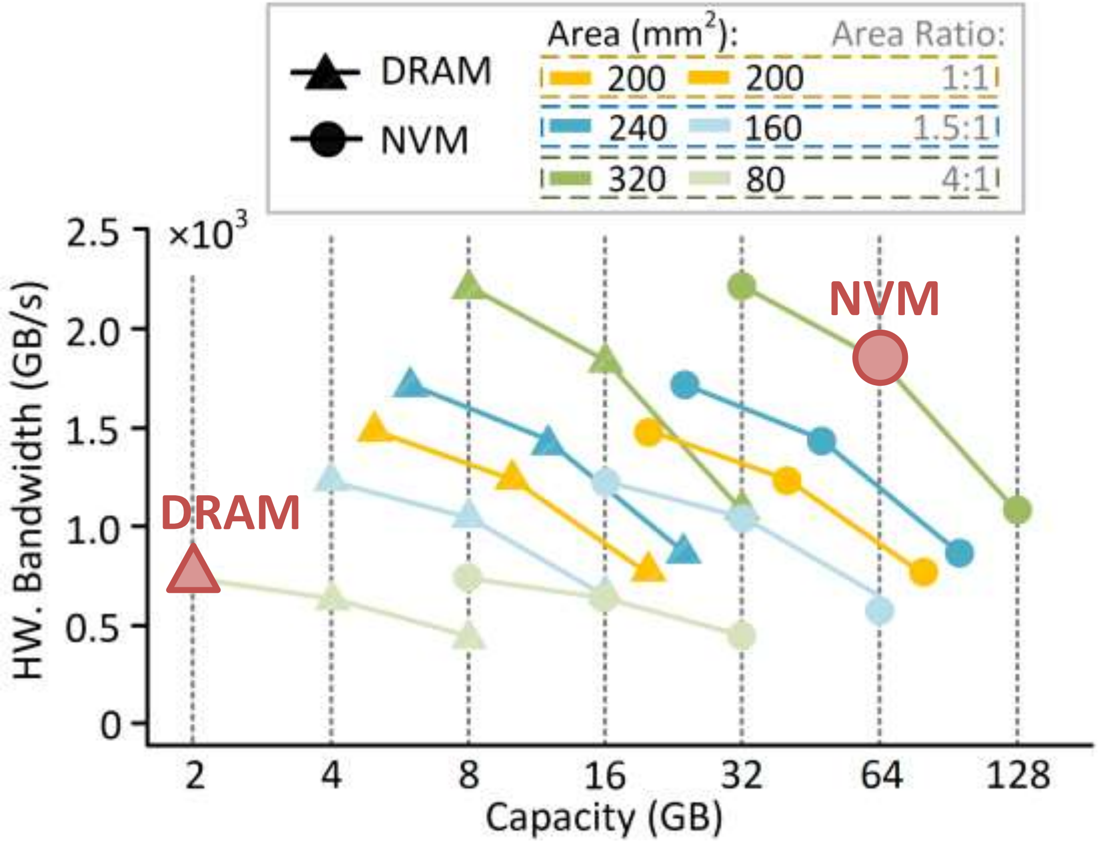
+

Analytical Model

Quickly estimates system/per-user throughput for each candidate configuration

Design Space Exploration Insight

- **Insight:** For large models with dense attention (e.g. MHA), an **approximately 4:1 NVM-to-DRAM area ratio** is usually optimal, with **NVM balancing capacity and bandwidth** and **DRAM maximizing bandwidth**
  - Weight and KVCache dominate runtime and **require both capacity and bandwidth**, so NVM receives more area budget
  - **NVM capacity benefit saturates** once on-chip buffering (**not batch size**) becomes the **data reuse bottleneck**; remaining area is better allocated to bandwidth
  - **DRAM should provide enough capacity for IA** and allocates the rest to bandwidth



*Data obtained from simulation using CACTI-3DD; only **part of the design space is shown** for demonstration

Outline

- Motivation
- SHyLA Design
  - 3D-Stacked NVM-DRAM Hybrid Memory Architecture Overview
  - Bandwidth-Utilization-Centric Dataflow
  - Two-Stage DSE
- **Evaluation**
- Takeaways

Evaluation Setup

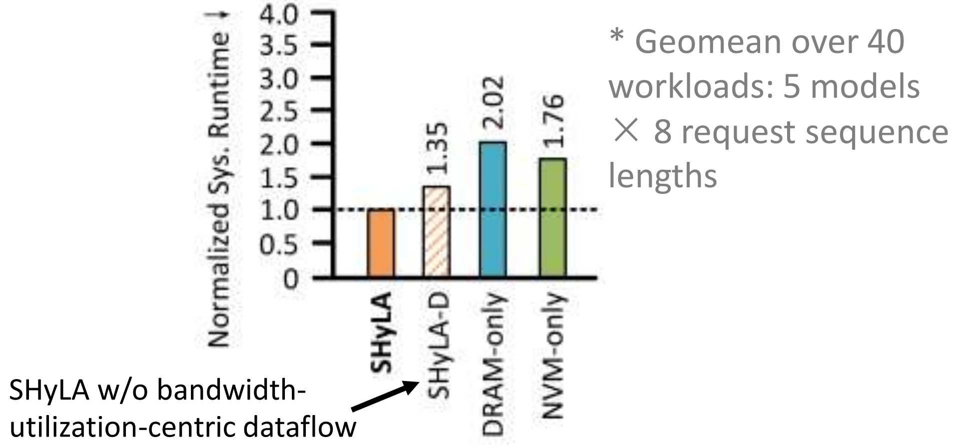
- **Simulator:** GPGPU-Sim[1] for overall performance + CACTI-3DD[2] for memory design + 3D-ICE[3] for thermal analysis
- **Evaluated Systems**
  - **GPU baselines:** NVIDIA H800 / AMD MI300x
  - **DSA baselines:** 3D-stacked **DRAM-only** system / 3D-stacked **NVM-only** system
  - **SHyLA** (3D-stacked NVM-DRAM hybrid system)
- **Benchmarks**
  - Model sizes range from 70B to 540B, model structure includes MHA/GQA and Dense/MoE
  - Request sequence lengths span 256 to 32k

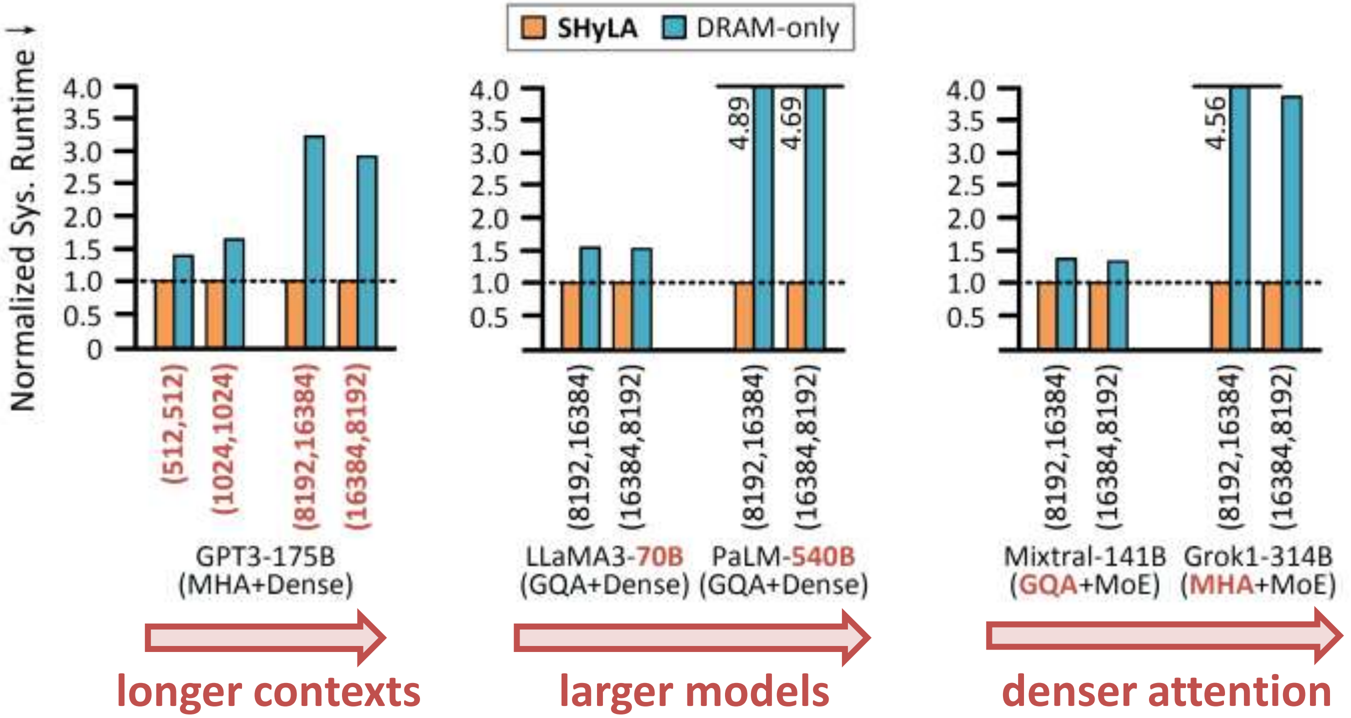
Model	Param.	MHA/GQA	Dense/MoE	L	H	d	$d_{\text{FFN}}/d_{\text{Expert}}$
LLaMA3	70B	GQA	Dense	80	64	8192	28672
GPT3	175B	MHA	Dense	96	96	12288	49152
PaLM	540B	GQA	Dense	118	48	18432	73728
Mixtral	141B	GQA	MoE	56	48	6144	16384
Grok1	314B	MHA	MoE	64	48	6144	32768

[1] Khairy et al., Accel-sim: An Extensible Simulation Framework for Validated GPU Modeling, ISCA 2020
[2] Chen et al., CACTI-3DD: Architecture-level Modeling for 3D Die-Stacked DRAM Main Memory, DATE 2012
[3] Sridhar et al., 3D-ICE: Fast Compact Transient Thermal Modeling for 3D ICs with Intertier Liquid Cooling, ICCAD 2010

Peak System Throughput

- Peak system throughput across sequence lengths, model sizes, and model structures





SHyLA achieves 2.02× and 1.76× geomean peak system throughput over DRAM-only and NVM-only systems

Dataflow

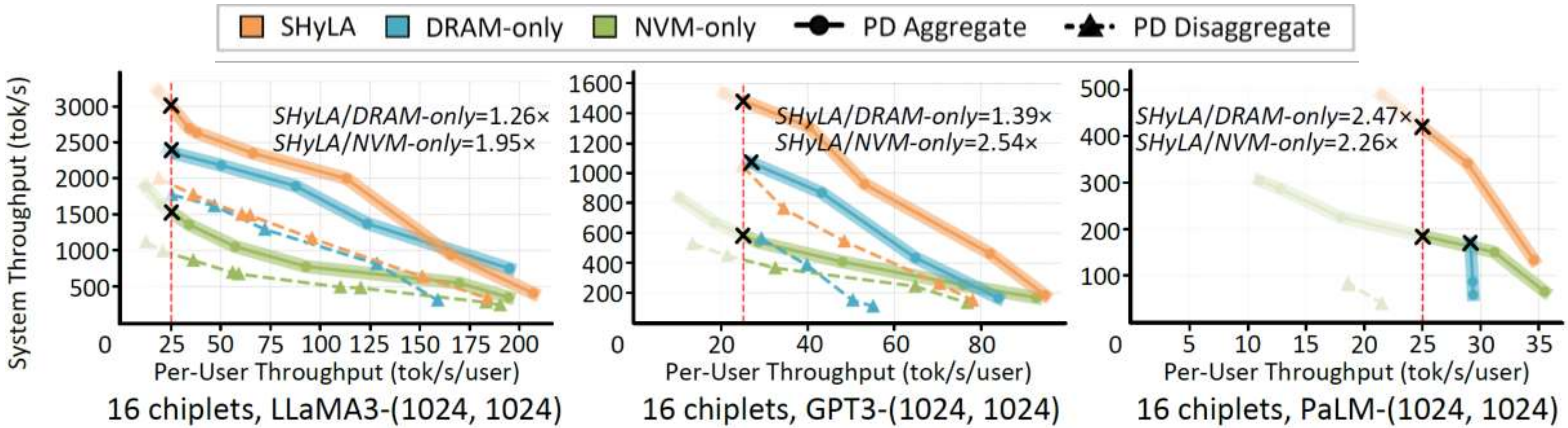
Bandwidth-utilization-centric dataflow contributes 1.35× by better exploiting 3D-stacked areal bandwidth

Advantageous Scenarios

SHyLA outperforms DRAM-only most when memory capacity is the bottleneck, as its larger capacity enables larger batches: long contexts / large models / dense attention

System Throughput Under Per-User SLO

- System throughput under a per-user SLO of 25 tok/s

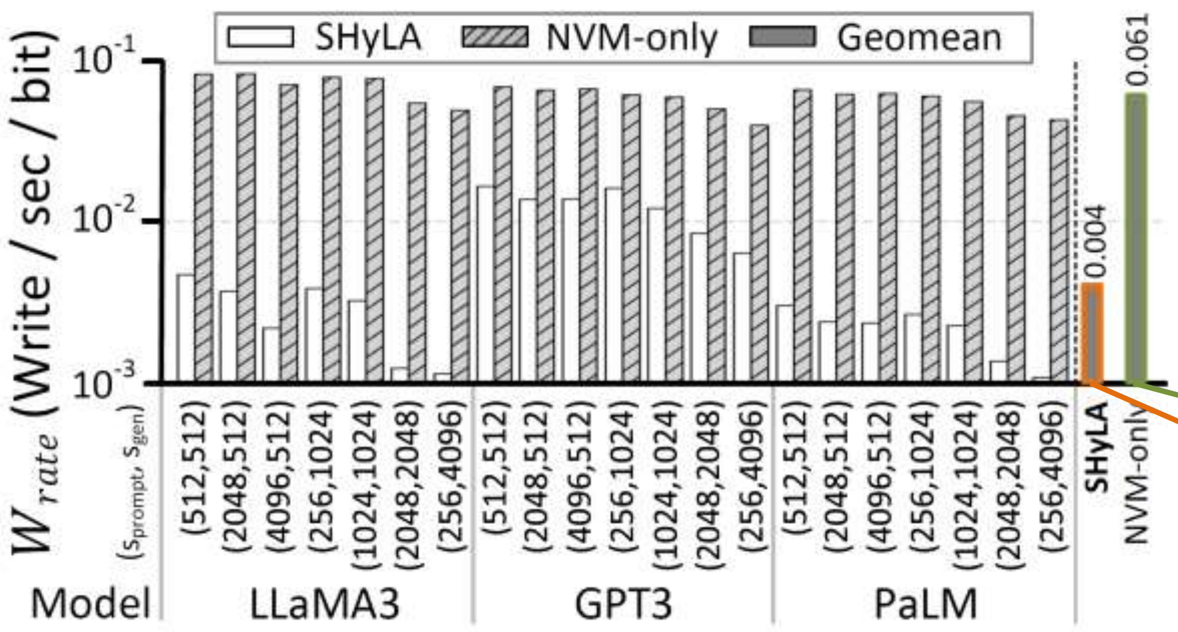


*PD disaggregation is omitted (SHyLA and DRAM-only) because Weight replication exceeds capacity

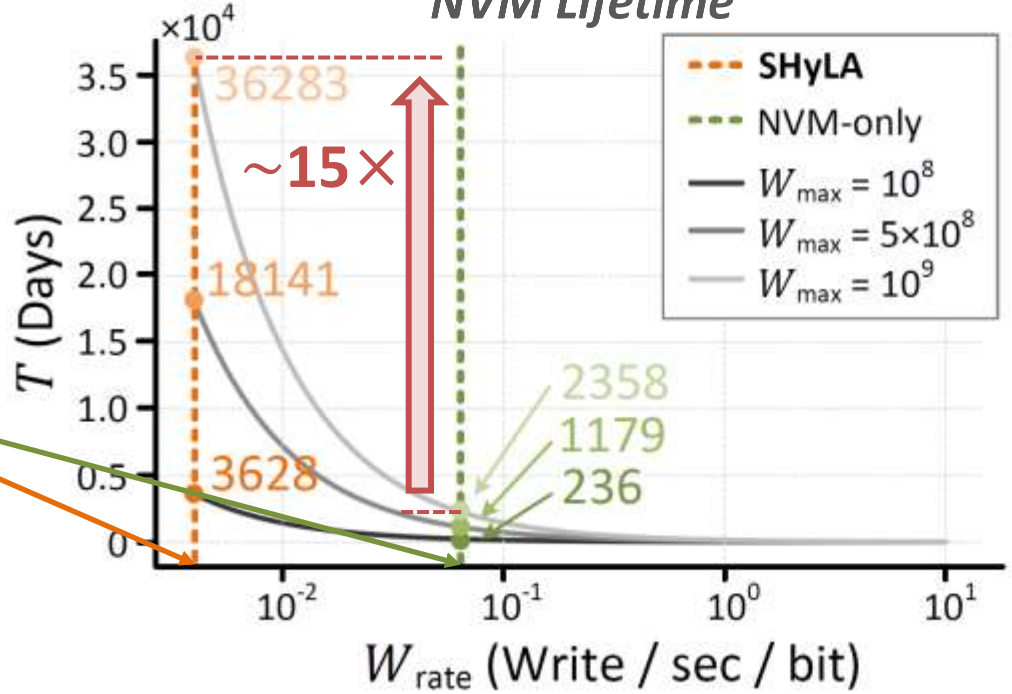
NVM Endurance

☐ NVM write rate and lifetime analysis

NVM Write Rate



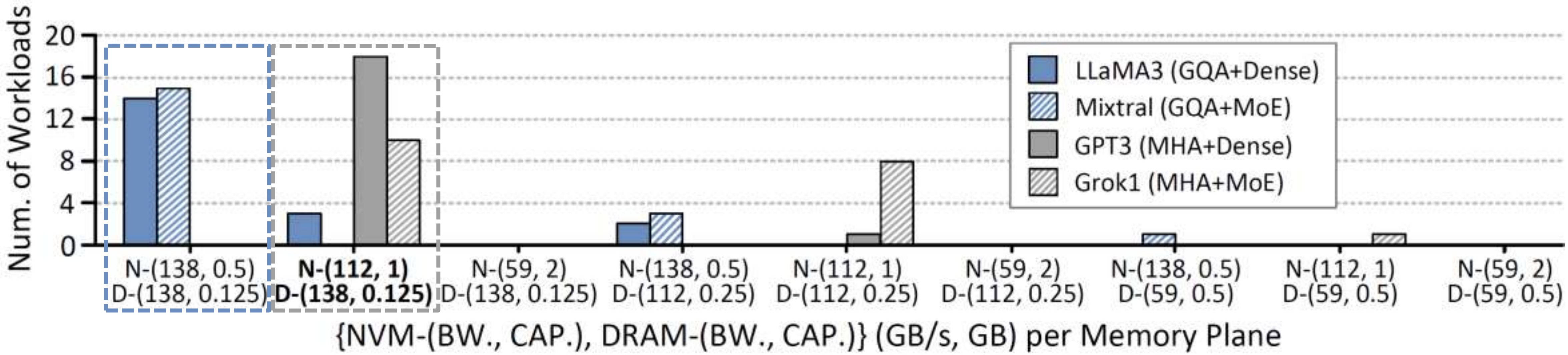
NVM Lifetime



DRAM absorbs the frequent write operations, extending the lifetime of **SHyLA** by **~15×**
(**~10 years** when endurance is 10^8) compared with an **NVM-only** system

DSE Analysis: Bandwidth–Capacity Trade-off Preference

- How different model structures prefer different bandwidth–capacity trade-offs in the NVM–DRAM hybrid memory system



Preference is determined by **attention sparsity**

Sparse attention (e.g. GQA)

lower KVCache pressure → larger batch → benefit of enlarging batch saturates

Favors NVM bandwidth

Dense attention (e.g. MHA)

higher KVCache pressure → smaller batch → enlarge batch for Weight reuse

Favors balanced NVM bandwidth–capacity

Outline

- Motivation
- SHyLA Design
  - 3D-Stacked NVM-DRAM Hybrid Memory Architecture Overview
  - Bandwidth-Utilization-Centric Dataflow
  - Two-Stage DSE
- Evaluation
- **Takeaways**

Takeaways

- This work proposes a **3D-stacked NVM-DRAM hybrid architecture**
  - Achieving 2.02× and 1.76× geomean peak system throughput over DRAM-only and NVM-only systems, respectively
- **Dual-heterogeneity match: align LLM data heterogeneity with NVM–DRAM device heterogeneity**
  - Weight → NVM; IA → DRAM; KVCache → NVM + DRAM
- **Effective bandwidth-capacity trade-off in 3D stacking translates NVM density into real bandwidth**, identifying an optimal hybrid memory design: **NVM balances capacity and bandwidth, while DRAM trades capacity for higher bandwidth (for large models with dense attention mechanism)**
  - Model attention sparsity determines memory bandwidth–capacity preferences
- Dataflow should be co-designed to convert **3D-stacked areal hardware bandwidth** into **effective bandwidth**

Thanks for your attention!